Final Project: Adaptive Soundscape

Jacob Jiang

Demo:

- https://adaptive-soundscape.vercel.app/

Code Snippet:

- Creating Realistic Wind Sound Using Web Audio API

<u>Web Audio API Terminology</u>

Nodes: Act as individual building blocks in the sound studio

Source Nodes: Create sound

Process Nodes: Modify sound (filters, gains, oscillators)

Destination: Speakers/Headphones

Code Snippet:

Set up the audio engine

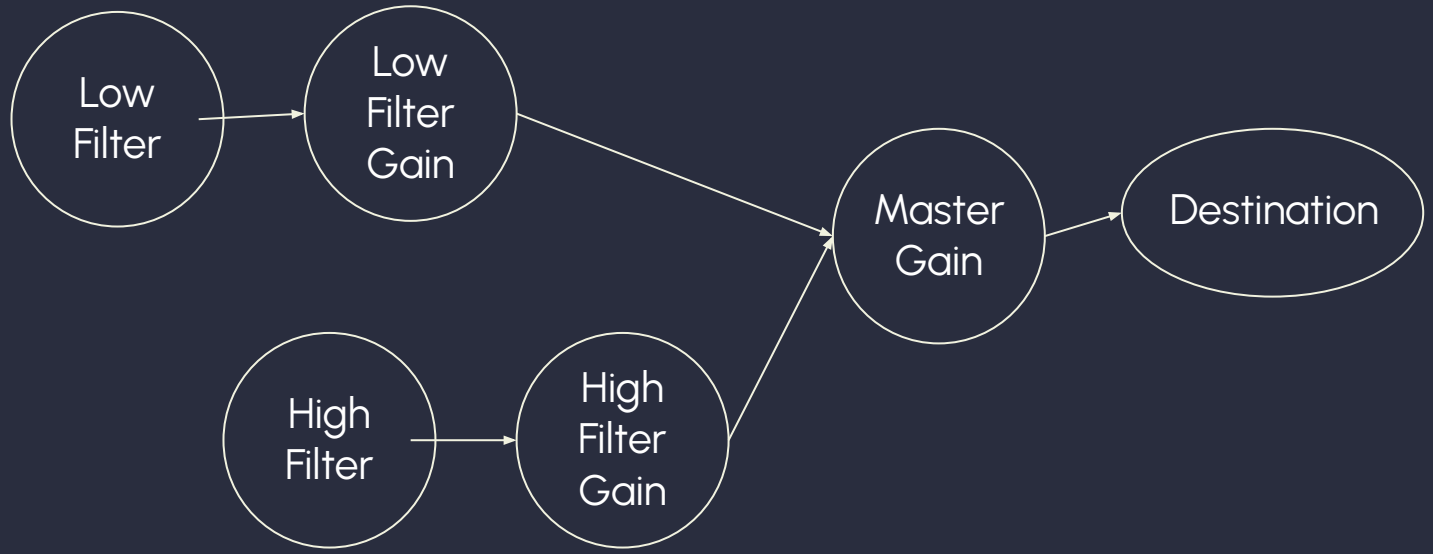
```
// attribute initialization for wind sound synthesis
this.ctx = new window.AudioContext()
this.source = null
this.masterGain = this.ctx.createGain()
this.lowFilterGain = this.ctx.createGain()
this.highFilterGain = this.ctx.createGain()
this.lowFilter = this.ctx.createBiquadFilter()
this.highFilter = this.ctx.createBiquadFilter()
this.highFilterLFO = this.ctx.createOscillator()
this.highFilterMod = this.ctx.createGain()
```

- createGain : Volume Control
- createBiquadFilter : Filter sound
- Both create nodes

this.[connectFrom].connect(this.[connectTo])

```
// 2-layer filtered noise -> masterGain (volume control) -> destination (speaker/headphone)
this.lowFilter.connect(this.lowFilterGain)
this.lowFilterGain.connect(this.masterGain)
this.highFilter.connect(this.highFilterGain)
this.highFilterGain.connect(this.masterGain)
this.masterGain.connect(this.ctx.destination)
```

Code Snippet:



Problem: No source!

Code Snippet:

Create white noise

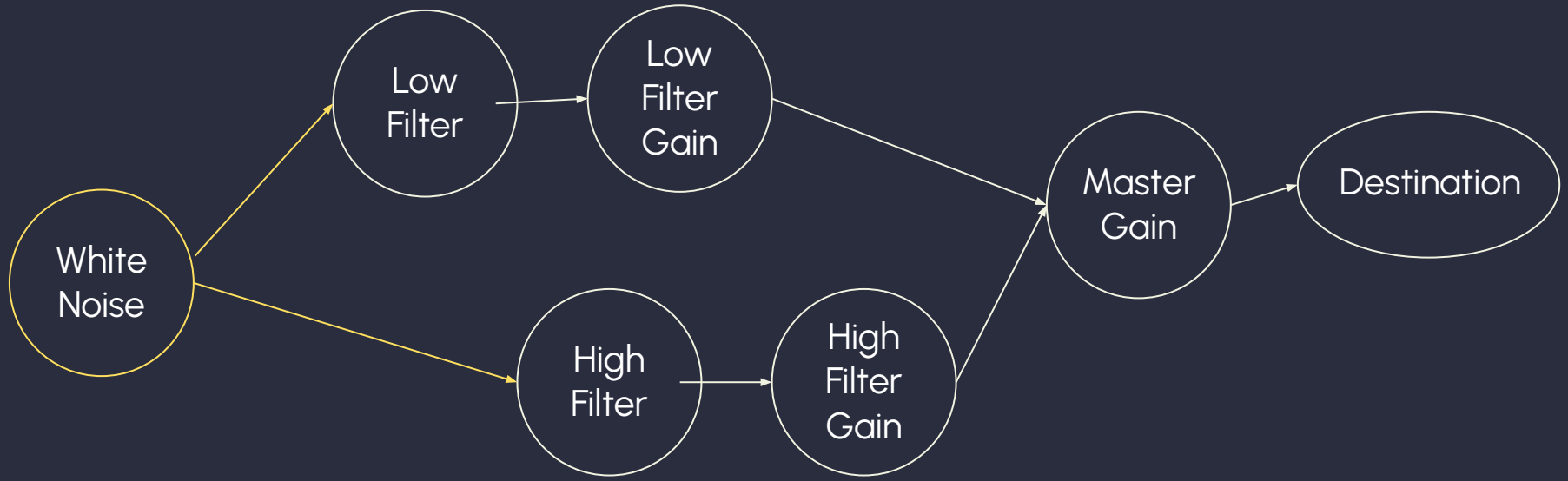
Create source as white noise

Connect source to filters and play noise

```
// fill buffer with random values to simulate static (wind)
for (let i = 0; i < bufferSize; i++) {
    output[i] = Math.random() * 2 - 1
}

// set up audio playback
this.source = this.ctx.createBufferSource()
this.source.buffer = this.noiseBuffer
this.source.loop = true

this.source.connect(this.lowFilter)
this.source.connect(this.highFilter)
this.source.start()

// fade in volume
this.masterGain.gain.linearRampToValueAtTime(0.3, this.ctx.currentTime + 2)
```

Code from startNoise function (called upon syncing weather)

Code Snippet:

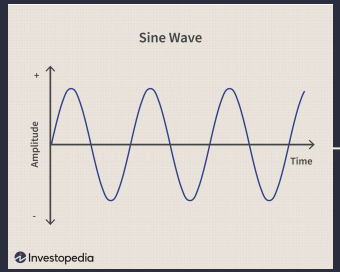


Problem: Flat Sound → unrealistic

Code Snippet:

- How to vary wind pitch?
  - Oscillators

- createGain - strength control



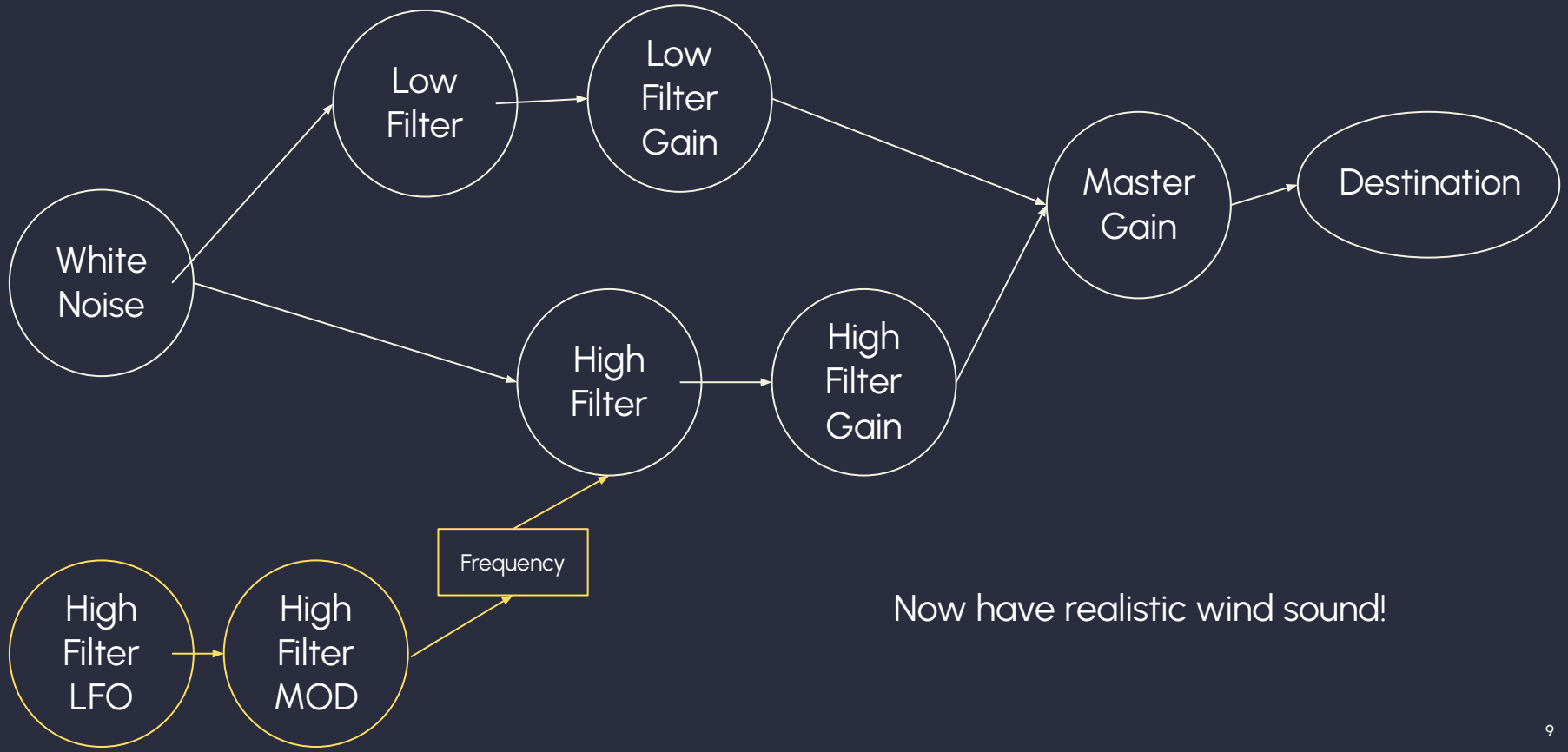

```
// attribute initialization for wind sound synthesis
this.ctx = new window.AudioContext()
this.source = null
this.masterGain = this.ctx.createGain()
this.lowFilterGain = this.ctx.createGain()
this.highFilterGain = this.ctx.createGain()
this.lowFilter = this.ctx.createBiquadFilter()
this.highFilter = this.ctx.createBiquadFilter()
this.highFilterLFO = this.ctx.createOscillator()
this.highFilterMod = this.ctx.createGain()
```

```
// setup low frequency oscillator (LFO) for wind variation
this.highFilterLFO.type = 'sine'
this.highFilterLFO.frequency.value = 0.2 // oscillation rate
this.highFilterMod.gain.value = 0 // initially no oscillation
```

Important: Connect to filter **frequency**

```
// connect LFO to high filter frequency
this.highFilterLFO.connect(this.highFilterMod)
this.highFilterMod.connect(this.highFilter.frequency)
```

Code Snippet:



Now have realistic wind sound!

Process:

Task	Tools Needed
Create Environmental Sounds	JavaScript (JS), Web Audio API
Fetch Weather At Current Location	JS, Open-Meteo API, Geolocation API
Play Sounds Based On Weather	HTML, Bootstrap CSS, JS
Update Visuals Based On Weather	HTML, Bootstrap CSS, JS
Add Manual Control for Sounds	HTML, Bootstrap CSS, JS

Issues Encountered:

- Creating realistic wind sounds by hand
- Dealing with delay from API calls
- Debugging when API calls weren't working as expected
- Trying to get familiar with utility-first CSS styling

Skills Learned:

- Familiarity with Web Audio API
- Connecting HTML to JS using IDs
- Manipulating HTML classes using JS for visual effect
- Handling API calls (fetch, await, parsing .json)
- Using Bootstrap CSS Grid (while aware of mobile-first)

Next Steps:

- Add more sound options (crickets, waterfall, etc.)
- Add volume dial for wind
- Add nighttime environment (sound and visual theme)
- Add more set weather conditions (thunderstorm, snow)

Final Project: Adaptive Soundscape

Jacob Jiang